



Deduction modulo theory

Gilles Dowek

► To cite this version:

Gilles Dowek. Deduction modulo theory. All about proofs. Proofs for all., Jul 2014, Wien, Austria. ⟨hal-01101829⟩

HAL Id: hal-01101829

<https://inria.hal.science/hal-01101829v1>

Submitted on 26 Jan 2015

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.



Distributed under a Creative Commons CC BY-SA 4.0 - Attribution - ShareAlike - International License

Deduction modulo theory

Gilles Dowek

Inria, 23 avenue d'Italie, CS 81321, 75214 Paris Cedex 13, France.

`gilles.dowek@inria.fr`

1 Introduction

1.1 Weaker vs. stronger systems

Contemporary proof theory goes into several directions at the same time. One of them aims at analyzing proofs, propositions, connectives, etc., that is at decomposing them into more atomic objects. This often leads to design systems that are weaker than Predicate logic, but that have better algebraic or computational properties, and to try to reconstruct part of Predicate logic on top of these systems. Propositional logic, linear logic, deep inference, equational logic, explicit substitution calculi, etc. are examples of such systems. From this point of view, Predicate logic appears more as the ultimate goal of the journey, than as its starting point.

Another direction considers that very little can be expressed in pure Predicate logic and that stronger systems are needed, for instance to express genuine mathematical proofs. Axiomatic theories, modal logics, type theories, etc. are examples of such systems that are more expressive than pure Predicate logic. There, Predicate logic is the starting point of the journey.

Although both points of view coexist in many research projects, these two approaches to proof theory often lead to different systems and different problems.

Deduction modulo theory is part of the second group, as it focuses on proofs in theories. The concern of integrating theories to proof theory is that of several research groups. See, for instance, [52] and [54] for related approaches.

1.2 Logical vs. theoretical systems

To design a system stronger than pure Predicate logic, several ways are possible. One is to extend Predicate logic with new logical operators, that is to design a logic, the second is to introduce function symbols and predicate symbols within Predicate logic and state axioms expressing the meaning of these symbols, that is to design a theory. The first approach can be illustrated by modal logics, the second by arithmetic or set theory. Simple type theory belongs to both groups as it can be defined either as a logic, in which case it is more often called *higher-order logic*, or as a theory in Predicate logic [32].

Deduction modulo theory is part of the second, theoretical rather than logical, group, as, like Predicate logic, it is a framework in which it is possible to define many theories.

1.3 Axioms vs. reduction rules

But, the main difference between Deduction modulo theory and the axiomatic approach is that a, in Deduction modulo theory, *theory* is not defined as a set of axioms, but as a set of reduction rules.

Indeed, axioms jeopardize most of the properties of proofs of pure Predicate logic. For instance, in pure Predicate logic, a constructive Natural deduction cut free proof always ends with an introduction rule, hence a constructive cut free existential proof always ends with an introduction rule of the existential quantifier. But this result does not extend to axiomatic theories, as a constructive cut free proof in a theory may also end, for instance, with the axiom rule.

In the same way, in automated theorem proving in pure Predicate logic, the search space of the proposition $\perp$ is always finite. But this result does not extend to axiomatic theories, that can generate an infinite search space for the proposition $\perp$.

To overcome these problems, axioms, in Deduction modulo theory, are replaced by sets of reduction rules. For instance, the axioms

$$\forall y (0 + y = y)$$

$$\forall x \forall y (S(x) + y = S(x + y))$$

are replaced by the reduction rules

$$0 + y \longrightarrow y$$

$$S(x) + y \longrightarrow S(x + y)$$

These reduction rules define a congruence $\equiv$ on propositions, and deduction is performed modulo this congruence. For instance, with the reduction rules above the propositions $2 + 2 = 4$ and $4 = 4$ are congruent, hence any proof of the latter is a proof of the former. If, to define equality, we add the following rules [1], which directly rewrite atomic propositions

$$0 = 0 \longrightarrow \top$$

$$S(x) = 0 \longrightarrow \perp$$

$$0 = S(y) \longrightarrow \perp$$

$$S(x) = S(y) \longrightarrow x = y$$

then the proposition $2 + 2 = 4$ and $\top$ are congruent, and any proof of $\top$ —for instance the mere application of the introduction rule for $\top$ —is a proof of the proposition $2 + 2 = 4$

$$\frac{}{\vdash 2 + 2 = 4} \top\text{-intro}$$

1.4 Deduction vs. computation

In the example above, the proposition $2 + 2 = 4$ is provable because it reduces to $\top$. More generally, all propositions that reduce to $\top$ are provable. But the converse is not true: not all provable propositions reduce to $\top$. Indeed, reducibility to $\top$ is often a decidable property, while provability is not.

On the contrary, the fact that the proposition $2 + 2 = 4$ has a trivial proof, because it reduces to $\top$, shows that the truth of this proposition rests on a mere computation and not on a genuine deduction.

Thus, Deduction modulo theory also permits one to distinguish the computation part from the deduction part within a proof, whereas Predicate logic flattens computation and deduction at the same level.

1.5 The origins of Deduction modulo theory

Deduction modulo theory was first introduced in the area of automated theorem proving.

Indeed, in Resolution, or in other automated theorem proving methods, instead of using equational axioms, for instance the associativity axiom, we often replace standard unification with equational unification, for instance unification modulo associativity [57]. In the same way, in Simple type theory, instead of using the β -conversion axiom, we replace standard unification with equational unification modulo β -equivalence: higher-order unification [2, 48, 49]. The automated theorem proving method obtained this way is called *Equational resolution*.

A way to prove the soundness and completeness of Equational resolution is to introduce a Natural deduction system, or a Sequent calculus system, where propositions are identified modulo associativity, or modulo β -equivalence. Then, this system can be proved to be equivalent to the axiomatic presentation of the theory. Finally, the soundness and completeness of Equational resolution are proved relatively to this system, where every deduction step is performed modulo the congruence.

So Deduction modulo theory comes from automated theorem proving. But it was soon understood that this idea of identifying propositions modulo a congruence was also the idea behind the notion of definitional equality in Martin-Löf's Intuitionistic type theory [53] and that Deduction modulo theory could also be seen as an extension of this notion of definitional equality to Predicate logic.

Another source of inspiration is the extension of Natural deduction with folding and unfolding rules, introduced by Prawitz [58, 23, 43, 40, 24]. In this system, it is not possible to identify an atomic proposition P with a proposition A . But, it is possible to introduce non logical deduction rules

$$\frac{A}{P} \qquad \frac{P}{A}$$

The relation between the two frameworks is detailed in [28].

2 Proof Systems

The idea of reasoning modulo a theory can be used in different formalisms: Natural deduction, Sequent calculus, λ -calculus, etc. Thus, Deduction modulo theory exists in many flavors.

2.1 Natural Deduction modulo theory

Let us start with constructive Natural deduction. The rules of constructive Natural deduction modulo theory are obtained by transforming the rules of constructive Natural deduction, to allow to use of the congruence. For instance, the rule

$$\frac{\Gamma \vdash A \Rightarrow B \quad \Gamma \vdash A}{\Gamma \vdash B} \Rightarrow\text{-elim}$$

is transformed into

$$\frac{\Gamma \vdash C \quad \Gamma \vdash A \Rightarrow\text{-elim}}{\Gamma \vdash B} \text{ if } C \equiv (A \Rightarrow B)$$

where the proposition $A \Rightarrow B$ is replaced by any congruent proposition C . Applying the same transformation to all Natural deduction rules yields the system of Figure 1.

For instance, consider the congruence defined by the *subset reduction rule*

$$x \subseteq y \longrightarrow \forall z (z \in x \Rightarrow z \in y)$$

The sequent $\vdash s \subseteq s$ has the proof

$$\frac{\frac{\frac{}{z \in s \vdash z \in s} \text{ axiom}}{\vdash z \in s \Rightarrow z \in s} \Rightarrow\text{-intro}}{\vdash s \subseteq s} \langle z, z \in s \Rightarrow z \in s \rangle \forall\text{-intro}$$

Note that when two propositions A and B are provably equivalent, that is when $A \Leftrightarrow B$ is provable, then the proposition A has a proof if and only if the proposition B has a proof, but the propositions A and B need not have the same proofs. In contrast, when two propositions are congruent, that is when $A \equiv B$, then every proof of A is a proof of B and vice versa, thus the propositions A and B have the same proofs.

Sequent calculus modulo theory can be defined in the same way: the rule

$$\frac{\Gamma \vdash A \quad \Gamma, B \vdash \Delta}{\Gamma, A \Rightarrow B \vdash \Delta} \Rightarrow\text{-left}$$

for instance, is transformed into

$$\frac{\Gamma \vdash A \quad \Gamma, B \vdash \Delta}{\Gamma, C \vdash \Delta} \Rightarrow\text{-left} \text{ if } C \equiv (A \Rightarrow B)$$

$$\begin{array}{c}
\text{axiom} \\
\hline
\Gamma, A \vdash B \text{ if } A \equiv B
\end{array}$$

$$\begin{array}{c}
\text{T-intro} \\
\hline
\Gamma \vdash A \text{ if } A \equiv \top
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash B \quad \perp\text{-elim} \\
\hline
\Gamma \vdash A \text{ if } B \equiv \perp
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash A \quad \Gamma \vdash B \quad \wedge\text{-intro} \\
\hline
\Gamma \vdash C \text{ if } C \equiv (A \wedge B)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash C \quad \wedge\text{-elim} \\
\hline
\Gamma \vdash A \text{ if } C \equiv (A \wedge B)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash C \quad \wedge\text{-elim} \\
\hline
\Gamma \vdash B \text{ if } C \equiv (A \wedge B)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash A \quad \vee\text{-intro} \\
\hline
\Gamma \vdash C \text{ if } C \equiv (A \vee B)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash D \quad \Gamma, A \vdash C \quad \Gamma, B \vdash C \quad \vee\text{-elim} \\
\hline
\Gamma \vdash C \text{ if } D \equiv (A \vee B)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash B \quad \vee\text{-intro} \\
\hline
\Gamma \vdash C \text{ if } C \equiv (A \vee B)
\end{array}$$

$$\begin{array}{c}
\Gamma, A \vdash B \quad \Rightarrow\text{-intro} \\
\hline
\Gamma \vdash C \text{ if } C \equiv (A \Rightarrow B)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash C \quad \Gamma \vdash A \quad \Rightarrow\text{-elim} \\
\hline
\Gamma \vdash B \text{ if } C \equiv (A \Rightarrow B)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash A \quad \langle x, A \rangle \quad \forall\text{-intro} \\
\hline
\Gamma \vdash B \text{ if } B \equiv (\forall x A) \\
\text{and } x \notin FV(\Gamma)
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash B \quad \langle x, A, t \rangle \quad \forall\text{-elim} \\
\hline
\Gamma \vdash C \text{ if } B \equiv (\forall x A) \text{ and } C \equiv [t/x]A
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash C \quad \langle x, A, t \rangle \quad \exists\text{-intro} \\
\hline
\Gamma \vdash B \text{ if } B \equiv (\exists x A) \text{ and } C \equiv [t/x]A
\end{array}$$

$$\begin{array}{c}
\Gamma \vdash C \quad \Gamma, A \vdash B \quad \langle x, A \rangle \quad \exists\text{-elim} \\
\hline
\Gamma \vdash B \text{ if } C \equiv (\exists x A) \\
\text{and } x \notin FV(\Gamma B)
\end{array}$$

Fig. 1. Natural Deduction Modulo Theory

where the proposition $A \Rightarrow B$ is replaced by any proposition C such that $C \equiv (A \Rightarrow B)$. And the other rules are transformed alike. See, for instance, [38] for a description of the full system.

Another variant of Natural deduction modulo theory and Sequent calculus modulo theory is *Super-deduction* [62, 18]. In Super-deduction, new deduction rules are computed from the reduction rules. For instance, the subset reduction rule yields the deduction rules

$$\frac{\Gamma, z \in x \vdash z \in y}{\Gamma \vdash x \subseteq y} z \notin FV(\Gamma)$$

$$\frac{\Gamma \vdash x \subseteq y \quad \Gamma \vdash z \in x}{\Gamma \vdash z \in y}$$

These rules are closer to the informal mathematical style than, for instance, Natural deduction rules. Indeed, to prove $x \subseteq y$, we often consider a generic element in x and prove that it is in y without using the universal quantifier and the implication of the proposition $\forall z (z \in x \Rightarrow z \in y)$. The fact that these derived rules use atomic propositions only also explains why connectives and quantifiers are less often used in informal proofs than in formal ones.

2.2 Polarized deduction modulo theory

In Natural deduction modulo theory and in Sequent calculus modulo theory, the reduction rules are just used to define the congruence $\equiv$. In fact, this congruence does not even need to be defined with reduction rules and it could be any congruence, provided it is decidable and it does not identify non-atomic propositions with different head symbols. But we may also want to stress that computation is oriented and take, in these rules, the condition $C \longrightarrow^* (A \Rightarrow B)$ instead of $C \equiv (A \Rightarrow B)$, meaning that in the sequent $\Gamma, C \vdash \Delta$, the proposition C can only be reduced.

In particular, the axiom rule

$$\frac{}{\Gamma, A \vdash B} \text{ axiom if } A \equiv B$$

would be restated

$$\frac{}{\Gamma, A \vdash B} \text{ axiom if } A \longrightarrow^* C \text{ and } B \longrightarrow^* C$$

If the theory contains rewrite rules on terms only, and t and u are two terms such that $t \equiv u$, it is still possible to prove the sequent $P(t) \vdash P(u)$. But when t and u do not have a common reduct, the proof of $P(t) \vdash P(u)$ contains cuts. In other words, in this particular case, the Sequent calculus modulo theory has the cut elimination property if and only if the reduction system is confluent [30] and Newman's algorithm [55]—which permits transforming an equational proof into a valley proof—is a cut-elimination algorithm.

This idea of using a rewrite relation rather than a congruence in the deduction rules can be developed further: the subset reduction rule permits to prove the equivalence

$$x \subseteq y \Leftrightarrow \forall z (z \in x \Rightarrow z \in y)$$

Thus, when the atomic proposition P reduces to the proposition A , P and A must be equivalent. For instance, it is not possible to reduce $Equilateral(x)$ to $Isosceles(x)$ because a triangle may be isosceles without being equilateral.

More generally, it is easy to transform an axiom of the form $P \Leftrightarrow A$ into a reduction rule $P \rightarrow A$, but, although it is possible [17], it is not easy to transform an axiom of the form $P \Rightarrow A$ into a reduction rule. We want to replace such an axiom with a rule that permits reducing P into A when P is a hypothesis, but not when it is a goal.

This leads to an extension of Deduction modulo theory, called *Polarized deduction modulo theory* where reduction rules are classified into positive and negative, the positive rules may apply to the positive occurrences of atomic propositions and the negative ones to the negative occurrences.

For instance, in Polarized sequent calculus modulo theory, the left rule of the implication is stated

$$\frac{\Gamma \vdash A \quad \Gamma, B \vdash \Delta}{\Gamma, C \vdash \Delta} \Rightarrow\text{-left} \quad \text{if } C \rightarrow_{-}^{*} (A \Rightarrow B)$$

and its right rule

$$\frac{\Gamma, A \vdash B}{\Gamma \vdash C} \Rightarrow\text{-right} \quad \text{if } C \rightarrow_{+}^{*} (A \Rightarrow B)$$

Polarized deduction modulo theory is the flavor of Deduction modulo theory that is more often used in automated theorem proving.

The first reason is that, in clause based theorem proving, a reduction rule of the form

$$x \in y \cup z \rightarrow x \in y \vee x \in z$$

can be used to reduce a positive literal in a clause but not a negative one. For instance, the clause $L_1 \vee L_2 \vee a \in b \cup c$ reduces to the clause $L_1 \vee L_2 \vee a \in b \vee a \in c$, but the clause $L_1 \vee L_2 \vee \neg a \in b \cup c$ reduces to the proposition $L_1 \vee L_2 \vee \neg(a \in b \vee a \in c)$ that is not a clause. In contrast, if we replace this reduction rule by the polarized rules

$$x \in y \cup z \rightarrow_{-} x \in y \vee x \in z$$

$$x \in y \cup z \rightarrow_{+} x \in y$$

$$x \in y \cup z \rightarrow_{+} x \in z$$

then the clause $L_1 \vee L_2 \vee \neg a \in b \cup c$ reduces to the clauses $L_1 \vee L_2 \vee \neg a \in b$ and to $L_1 \vee L_2 \vee \neg a \in c$. More generally, any reduction system can be transformed this way to a clausal one [42].

The second reason is that any consistent set of axioms can be transformed into a Polarized reduction system that is classically equivalent [29, 14] and some sets of axioms can be transformed into a Polarized reduction system that is constructively equivalent [11].

Interestingly, this result has been proved with applications to automated theorem proving in mind, it uses automated theorem proving methods, but it is a purely proof-theoretical result.

2.3 Expressing theories in Deduction modulo theory

The early work on expressing theories in Deduction modulo theory was focused on specific theories: Simple type theory [34], Arithmetic [39, 1], Set theory [37], etc.

Then, as already said, systematic ways of transforming sets of axioms into sets of reduction rules have been investigated [29, 14, 11].

2.4 The $\lambda\Pi$ -calculus modulo theory

The early developments of Deduction modulo theory were independent of the proofs-as-algorithms paradigm, also known as the Brouwer-Heyting-Kolmogorov interpretation, that is the idea that a proof of $A \Rightarrow B$, for instance, is an algorithm transforming proofs of A into proofs of B . In Deduction modulo theory, like in Predicate logic, terms, propositions, and proofs belong to three different languages, and proofs are not terms. But we have mentioned that one of the origins of Deduction modulo theory was the definitional equality of Martin-Löf's Intuitionistic type theory. This suggests that this idea of identifying congruent propositions can also be useful in systems based on the proofs-as-algorithms paradigm.

The simplest system to express proofs of Predicate logic as algorithms is the λ -calculus with dependent types [47], also known as the $\lambda\Pi$ -calculus. This leads to the development of an extension of the $\lambda\Pi$ -calculus, called the $\lambda\Pi$ -calculus modulo theory [22]. This system is closely related to Martin-Löf's logical framework [56].

Any theory that can be expressed in minimal Deduction modulo theory, that is in the restriction of Deduction modulo theory, where the only logical operators are the implication and the universal quantifier, can be expressed in the $\lambda\Pi$ -calculus modulo theory. In particular Simple type theory can be expressed in the $\lambda\Pi$ -calculus modulo theory. An interesting point here is that the Calculus of Constructions [20] has been designed to express proofs of Simple type theory as algorithms. It happens that $\lambda\Pi$ -calculus modulo theory also can express those proofs as algorithms. This suggests that the Calculus of Constructions itself could be expressed in the $\lambda\Pi$ -calculus modulo theory, and this is indeed the case [22]. The embedding of the Calculus of Constructions into the $\lambda\Pi$ -calculus modulo theory follows closely the expression of Simple type theory in Deduction modulo theory.

It happens *a posteriori* that this embedding of the Calculus of Constructions into the $\lambda\Pi$ -calculus modulo theory can be seen as an extension of the $\lambda\Pi$ -calculus with an impredicative universe *à la* Tarski [3] and thus that there is a strong link between the expression of Simple type theory in Predicate logic and the notion of universe *à la* Tarski.

3 Properties

3.1 Models

The usual models of classical Predicate logic, valued in $\{0, 1\}$, can be used for Deduction modulo theory. A congruence $\equiv$ is said to be valid in a model when $A \equiv B$ implies $\llbracket A \rrbracket_\phi = \llbracket B \rrbracket_\phi$ for all valuations ϕ , and a soundness and completeness theorem can be proved using standard methods.

Like for Predicate logic, the set of truth values $\{0, 1\}$ can be extended to any Boolean algebra, allowing to prove a stronger completeness theorem: given a theory, there exists a model such that the propositions valid in this model are exactly the propositions provable in this theory.

Boolean algebras can be extended to Heyting algebras to define a sound and complete semantics for constructive logic.

However, in all these models—valued in $\{0, 1\}$, in Boolean algebras and in Heyting algebras—, two provably equivalent propositions always have the same truth value: if $A \Leftrightarrow B$ is valid, then $A \Rightarrow B$ and $B \Rightarrow A$ are valid, hence $\llbracket A \rrbracket_\phi \leq \llbracket B \rrbracket_\phi$ and $\llbracket B \rrbracket_\phi \leq \llbracket A \rrbracket_\phi$ and by antisymmetry $\llbracket A \rrbracket_\phi = \llbracket B \rrbracket_\phi$. Thus, there is no way to make a difference, in the model, between provable equivalence and congruence: whether A and B are just equiprovable or have the same proofs, they have the same truth value.

A way to overcome this is to extend Boolean algebras and Heyting algebras by dropping the antisymmetry condition on the relation $\leq$. This relation is then a pre-order and the algebras defined this way can be called *pre-Boolean* algebras [10] and *pre-Heyting* algebras [31]. The soundness theorem is proved exactly the same way—antisymmetry is never used in this proof—, and the completeness is simpler as the class of models is larger. This corresponds to the intuition that the relation $\leq$, defined by $A \leq B$ if $A \Rightarrow B$ is provable, is reflexive and transitive, but not antisymmetric.

This way, two provably equivalent propositions may be interpreted by distinct truth values, unlike two congruent propositions that must be interpreted by the same one, and it is possible to define models where a proposition A is interpreted by the set of its proofs.

When a theory has a model valued in some pre-Heyting algebra it is consistent, when it has a model valued in all pre-Heyting algebras it is said to be *super-consistent*.

3.2 Cut-elimination

Proof-reduction is defined in Deduction modulo theory in the same way as in Predicate logic, but the difference is that it does not always terminate. Indeed, if we define a theory with the reduction rule $P \longrightarrow (P \Rightarrow Q)$ the sequent $\vdash Q$

has the following proof

$$\begin{array}{c}
\frac{\overline{P \vdash P \Rightarrow Q} \text{ axiom} \quad \overline{P \vdash P} \text{ axiom}}{P \vdash Q} \Rightarrow\text{-elim} \quad \frac{\overline{P \vdash P \Rightarrow Q} \text{ axiom} \quad \overline{P \vdash P} \text{ axiom}}{P \vdash Q} \Rightarrow\text{-elim} \\
\frac{P \vdash Q}{\vdash P \Rightarrow Q} \Rightarrow\text{-intro} \quad \frac{P \vdash Q}{\vdash P} \Rightarrow\text{-intro} \\
\hline
\vdash Q \quad \Rightarrow\text{-elim}
\end{array}$$

that contains a cut and that reduces to itself.

Moreover, it is possible to prove that all cut free, that is irreducible, proofs end with an introduction rule, thus not only this proof does not terminate, but the sequent $\vdash Q$ has no cut free proof.

And a similar example can be built with a terminating reduction system [38].

Not only some theories have the cut-elimination property and some others do not, but this property is even undecidable [17, 46].

Thus, unlike for axiomatic theories, the notion of proof-reduction can be defined in a generic, theory independent, way, and the properties of cut free proofs, such as the property that the last rule of a cut free proof is an introduction rule can be proved in a generic way. But, the proof-termination theorem itself must be proved for each theory.

Using a method introduced to prove the termination of proof reduction in Simple type theory [41], we can prove that proof-reduction terminates in some theory, if a reducibility candidate $\llbracket A \rrbracket$ can be associated to each proposition A , in such a way that two congruent propositions are associated with the same reducibility candidate [38]

$$A \equiv B \text{ implies } \llbracket A \rrbracket = \llbracket B \rrbracket$$

This association of a reducibility candidate to each proposition is thus a model valued in the algebra of the reducibility candidates and the condition that two congruent propositions are associated with the same reducibility candidate is the validity of this congruence in this model.

This way, we get that if a theory has a model valued in the algebra of reducibility candidates, then proof-reduction strongly terminate.

The algebra of reducibility candidates is a pre-Heyting algebra—but not a Heyting algebra—thus we also get that proof-reduction terminates in super-consistent theories.

This semantic view on termination of proof reduction theorems also permits to relate these termination proofs to the so called *semantic* cut-elimination proofs that proceed by proving a completeness result for cut free provability. First, without proving the termination of proof-reduction, it is possible to prove directly that, in a super-consistent theory, each provable proposition has a cut free proof [36, 10]. This completeness proof does not use the pre-Heyting algebra of reducibility candidates but a simpler one.

Then, in some non super-consistent theories, proof reduction does not terminate, but each provable proposition has a cut free proof [44]. An example is obtained by replacing the proposition Q by $\top$ in the example above. This proof still fails to terminate but the sequent $\vdash \top$ has another proof, that is

cut free. Such cut-elimination theorems can only be proved via a completeness theorem and, when they are proved constructively, the constructive content of these proofs is a proof-transformation algorithm, that need not be related to proof-reduction.

Finally, some theories do not have the cut elimination property, but they can sometimes be extended to theories that have this property by adding derivable reduction rules [17, 15]. This saturation process can be compared to Knuth-Bendix method [51]—remember that confluence is a special case of cut-elimination—that does not prove that all reduction systems are confluent, but that, in some cases, it is possible to extend a reduction system with derivable rules, to make it confluent.

3.3 Automated theorem proving methods

Deduction modulo theory has been introduced to design and study automated theorem proving methods. The first method introduced was a variant of Resolution [35] that was too complicated because rules were not polarized. Thus, clauses could rewrite to non clausal propositions that needed to be dynamically transformed into clausal form. Polarization permitted to simplify the method [33] and also to understand better its relation to other methods. This method is complete if and only if the theory has the cut-elimination property [45].

Imagine we have a clause

$$L_1 \vee L_2 \vee a \in b \cup c$$

and a negative reduction rule

$$x \in y \cup z \longrightarrow \neg x \in y \vee x \in z$$

then applying this rule to this clause yields the clause

$$L_1 \vee L_2 \vee a \in b \vee a \in c$$

But instead of this reduction rule, we could have taken a clause

$$\underline{\neg x \in y \cup z} \vee x \in y \vee x \in z$$

and Resolution, applied to the literal $a \in b \cup c$ and the underlined literal in the new clause, would have yielded the same result. Thus, there is no need to extend Resolution to handle reduction rules, but reduction rules can just be seen as special clauses, called *one-way clauses*. The Resolution rule cannot be applied to two one-way clauses and when it is applied to a one-way clause and an ordinary one, only the literal corresponding to the left-hand side of the reduction rule can be used. Thus, Polarized resolution modulo theory is just another variant of Equational resolution with clause restrictions—like the Set of support [63] and the Semantic resolution [61] strategies—and literal restrictions—like Ordered resolution.

But, unlike other variants of Resolution, its completeness is equivalent to a cut-elimination theorem. Thus, it permits to handle theories, such as Simple type theory, that cannot be handled, for instance, with Ordered resolution, as the completeness of Polarized resolution modulo the rules of Simple type theory implies cut elimination for Simple type theory and, unlike that of Ordered resolution, it cannot be proved in Simple type theory [16].

A side effect of this work is to show that, surprisingly, clause restriction strategies—such as the Set of support or the Semantic resolution strategy—and literal restriction strategies—such as Ordered resolution—can be combined, provided we do not consider theories that are just consistent, but theories that also have the cut elimination property.

These remarks also showed the way to combine this method with other selection strategies in Resolution. In particular, it has been shown that this restriction is compatible with Ordered resolution [12].

Besides Resolution, other proof-search methods have been investigated, in particular direct search in cut free sequent calculus modulo theory, also known as the *tableaux* method [9].

4 Implementations

The early work on Deduction modulo theory only led to experimental implementations. But more mature systems have been developed in the recent years.

4.1 Dedukti

Dedukti [6, 8, 59] is an implementation of the $\lambda\Pi$ -calculus modulo theory. It is thus based on the proofs-as-algorithms paradigm and proof-checking is reduced to type-checking. But type-checking itself may require an arbitrary amount of computation to check the congruence of two propositions.

Dedukti is a parametric system: by changing the reduction rules, we change the theory in which the proofs are checked. Thus Dedukti is a logical framework [47]. As the proofs of many different systems can be expressed in this framework Dedukti is mostly used to check proofs developed in other systems—hence its name: “to deduce” in Esperanto—: HOL [4], Focalize [19], Coq [7, 3], etc. as well as proofs produced by automated theorem proving systems, such as iProver, Zenon, iProver modulo, and Zenon modulo. The current goal of the project is to be able to interface proofs developed in different systems, and defining a standard for proofs in various theories, much the same way standards are defined, for instance, for SMT solvers [5, 60].

4.2 iProver modulo, Super Zenon and Zenon modulo

iProver modulo [13] is an implementation of Ordered polarized resolution modulo theory. It is developed as an extension of iProver. It has shown convincing experimental results compared to the axiomatic approach. A tool automatically

orienting axioms into rewriting systems usable by iProver Modulo is also available.

Super Zenon [50] is an implementation of Tableaux modulo theory specifically designed for a variant of Class theory—Second order logic—called *B set theory*, and using Super-deduction instead of the original Deduction modulo theory.

Zenon modulo [25, 26] is a generic implementation of the Tableaux modulo theory method. It comes with a heuristic that turns axioms into rewrite rules before performing proof-search, and also with a new hand-tailored expression of B set theory as a set of rewrite rules.

5 Trends and Open Problems

In recent years, the effort in Deduction modulo theory has been put on the development of implementations. In particular, we do not know how far we can go in interfacing proof systems using a logical framework such as Dedukti. We also need to investigate how having user defined reduction rules can impact tactic based proof development.

In automated theorem proving we do not understand yet how to mix Resolution modulo theory with equality specific methods such as superposition.

On the more proof-theoretical side, we know that super-consistency is a sufficient condition for the strong termination of proof reduction but we do not know if it is a necessary condition. As suggested in [21], the notion of super-consistency may require some adjustment so that we can prove that it is a necessary and sufficient condition for proof termination. Finally, some extension of Deduction modulo theory allow congruences that identify non-atomic propositions with different head-symbols [27], in particular isomorphic types such as $A \Rightarrow (B \wedge C)$ and $(A \Rightarrow B) \wedge (A \Rightarrow C)$, but we do not know yet how far we can go in this direction.

References

References

1. L. Allali. Algorithmic equality in Heyting arithmetic modulo. *Higher Order Rewriting*, 2007.
2. P.B. Andrews. Resolution in type theory. *The Journal of Symbolic Logic*, 36, 1971, pp. 414-432.
3. A. Assaf. A Calculus of Constructions with explicit subtyping. Manuscript, 2014.
4. A. Assaf. *Traduction de HOL en Dedukti*. Master thesis, 2012.
5. F. Besson, P. Fontaine, and L. Théry, A Flexible Proof Format for SMT: a Proposal *Proof Exchange for Theorem Proving*, 2011.
6. M. Boespflug. *Conception d'un noyau de vérification de preuves pour le lambda-Pi-calcul modulo*. Doctoral thesis, École polytechnique, 2011.
7. M. Boespflug and G. Burel. CoqInE: translating the calculus of inductive constructions into the lambda-pi-calculus modulo. *Proof Exchange for Theorem Proving*, CEUR Workshop Proceedings 878, 2012, pp. 44-50.

8. M. Boespflug, Q. Carbonneaux, and O. Hermant. The $\lambda\Pi$ -calculus Modulo as a Universal Proof Language. *Proof Exchange for Theorem Proving*, CEUR Workshop Proceedings 878, 2012, pp. 28-43.
9. R. Bonichon and O. Hermant. A semantic completeness proof for TaMeD. *LPAR*, Lecture Notes in Computer Science 4246, Springer, 2006, pp. 167-181.
10. A. Brunel, O. Hermant, and C. Houtmann. Orthogonality and Boolean Algebras for Deduction Modulo. *Typed Lambda Calculus and Applications*, Lecture Notes in Computer Science 6990, Springer, 2011, pp. 76-90.
11. G. Burel. Automating theories in intuitionistic logic. S. Ghilardi and R. Sebastiani (eds.), *FroCoS*, Lecture Notes in Artificial Intelligence 5749, Springer, 2009, pp. 181-197.
12. G. Burel. Embedding deduction modulo into a prover. A. Dawar and H. Veith (eds.), *CSL*, Lecture Notes in Computer Science 6247, Springer, 2010, pp. 155-169.
13. G. Burel. Experimenting with deduction modulo. V. Sofronie-Stokkermans and N. Bjørner (eds.), *CADE*, Lecture Notes in Computer Science 6803, Springer, 2011, pp. 162-176.
14. G. Burel. From Axioms to Rewriting Rules. Manuscript.
15. G. Burel. Cut Admissibility by Saturation. *RTA-TLCA*, Lecture Notes in Computer Science 8560, Springer, 2014.
16. G. Burel and G. Dowek. How can we prove that a proof search method is not an instance of another? *Fourth International Workshop on Logical Frameworks and Meta-Languages: Theory and Practice*. ACM International Conference Proceeding Series, 2009.
17. G. Burel and C. Kirchner. Regaining cut admissibility in deduction modulo using abstract completion. *Information and Computation*, 208(2), 2010, pp. 140-164.
18. P. Brauner, C. Houtmann, and C. Kirchner. Superdeduction at work. *Rewriting, Computation and Proof, Essays dedicated to Jean-Pierre Jouannaud on the occasion of his 60th birthday*, Lectures Notes in Computer Science 4600, Springer, 2007, pp. 132-166.
19. R. Cauderlier. *Traits orientés objet en $\lambda\Pi$ -calcul modulo : Compilation de FoCaLize vers Dedukti*. Master thesis, 2012.
20. T. Coquand and G. Huet. The calculus of constructions. *Information and Computation*, 76, 1988, pp. 95-120.
21. D. Cousineau. On completeness of reducibility candidates as a semantics of strong normalization. *Logical Methods in Computer Science*, 8(1), 2012.
22. D. Cousineau and G. Dowek. Embedding pure type systems in the lambda-Pi-calculus modulo. S. Ronchi Della Rocca, *Typed Lambda Calculi and Applications*, Lecture Notes in Computer Science 4583, Springer, 2007, pp. 102-117.
23. M. Crabbé. Non-normalisation de la théorie de Zermelo. Manuscript, 1974.
24. M. Crabbé. Stratification and cut-elimination. *The Journal of Symbolic Logic*, 56(1), 1991, pp. 213-226.
25. D. Delahaye, D. Doligez, F. Gilbert, P. Halmagrand, and O. Hermant, Zenon Modulo: When Achilles Outruns the Tortoise using Deduction Modulo. *Logic for Programming, Artificial Intelligence, and Reasoning*. Lecture Notes in Computer Science 8312, Springer, 2013, pp. 274-290.
26. D. Delahaye, D. Doligez, F. Gilbert, P. Halmagrand, O. Hermant. Proof Certification in Zenon Modulo: When Achilles Uses Deduction Modulo to Outrun the Tortoise with Shorter Steps. *International Workshop on the Implementation of Logics*, 2013.

27. A. Díaz-Caro and G. Dowek. Simply Typed Lambda-Calculus Modulo Type Isomorphism. Manuscript, 2014.
28. G. Dowek. About folding-unfolding cuts and cuts modulo. *Journal of Logic and Computation* 11(3), 2001, pp. 419-429.
29. G. Dowek. What is a theory? H. Alt, A. Ferreira (Eds.), *Symposium on Theoretical Aspects of Computer Science*, Lecture Notes in Computer Science 2285, Springer, 2002, pp. 50-64.
30. G. Dowek. Confluence as a cut elimination property. R. Nieuwenhuis (Ed.), *Rewriting Technique and Applications*, Lecture Notes in Computer Science 2706, Springer, 2003, pp 2-13.
31. G. Dowek. Truth values algebras and proof normalization. *TYPES 2006*, Lectures Notes in Computer Science 4502, Springer, 2007.
32. G. Dowek, Skolemization in Simple Type Theory: the Logical and the Theoretical Points of View, C. Benzmler, C. Brown, J. Siekmann and R. Statman (eds.), *Festschrift in Honour of Peter B. Andrews on his 70th Birthday*. College Publications, 2008.
33. G. Dowek. Polarized resolution modulo. *IFIP Theoretical Computer Science*, 2010.
34. G. Dowek, Th. Hardin, and C.Kirchner. HOL-lambda-sigma: an intentional first-order expression of higher-order logic. *Mathematical Structures in Computer Science*, 11, 2001, pp. 1-25.
35. G. Dowek, Th. Hardin, and C. Kirchner. Theorem proving modulo. *Journal of Automated Reasoning*, 31(1), 2003, pp. 33-72.
36. G. Dowek, O. Hermant. A Simple Proof that Super-Consistency Implies Cut Elimination. *Notre Dame Journal of Formal Logic*, 53(4), 2012, pp. 439-456.
37. G. Dowek and A. Miquel. Cut elimination for Zermelo's set theory. Manuscript.
38. G. Dowek and B. Werner. Proof normalization modulo. *The Journal of Symbolic Logic*, 68(4), 2003, pp. 1289-1316.
39. G. Dowek and B. Werner. Arithmetic as a theory modulo. J. Giesel (Ed.), *Term rewriting and applications*, Lecture Notes in Computer Science 3467, Springer, 2005, pp. 423-437.
40. J. Ekman. *Normal proofs in set theory*. Doctoral thesis, Chalmers university of technology and University of Göteborg, 1994.
41. J.-Y. Girard. Une extension de l'interprétation de Gödel à l'analyse et son application à l'élimination des coupures dans l'analyse et la théorie des types. J.E. Fenstad (Ed.) *Second Scandinavian Logic Symposium*, North-Holland, 1970.
42. Jianhua Gao. Clausal Presentation of Theories in Deduction Modulo. *Journal of Computer Science and Technology*, 28(6), 2013, pp. 1085-1096. DOI: 10.1007/s11390-013-1399-0.
43. L. Hallnäs. *On normalization of proofs in set theory*. Doctoral thesis, University of Stockholm, 1983.
44. O. Hermant. Semantic cut elimination in the Intuitionistic Sequent Calculus. *Typed Lambda Calculus and Applications*, Lecture Notes in Computer Science 3461, Springer, 2005, pp. 221-233.
45. O. Hermant. Resolution is cut-free. *Journal of Automated Reasoning*, 44(3), 2010, pp. 245-276.
46. O. Hermant. Personnel communication.
47. R. Harper, F. Honsell, G. Plotkin. A Framework for Defining Logics. *Proceedings of Logic in Computer Science*, 1987, pp. 194-204.
48. G. Huet. A mechanisation of Type Theory. *Third International Joint Conference on Artificial Intelligence*, 1973, pp. 139-146.

49. G. Huet. A Unification Algorithm for Typed λ -calculus. *Theoretical Computer Science*, 1, 1975, pp. 27-57.
50. M. Jacquél, K. Berkani, D. Delahaye, and C. Dubois. Tableaux Modulo Theories Using Superdeduction - An Application to the Verification of B Proof Rules with the Zenon Automated Theorem Prover. IJCAR, 2012, pp. 332-338.
51. D.E. Knuth and P.B. Bendix. Simple word problems in universal algebras. J. Leech (Ed.), *Computational Problems in Abstract Algebra*, Pergamon Press, 1970, pp. 263-297.
52. S. Negri and J. Von Plato. Cut elimination in the presence of axioms. *Bulletin of Symbolic Logic*, 4(4), 1998, pp. 418-435.
53. P. Martin-Löf. *Intuitionistic type theory*. Bibliopolis, 1984.
54. A. Naibo. *Le statut dynamique des axiomes: Des preuves aux modèles*. Doctoral thesis, 2013.
55. M.H.A. Newman. On theories with a combinatorial definition of “equivalence”. *Annals of Mathematics*, 43, 2, 1942, pp. 223-243.
56. B. Nordström, K. Petersson, and J.M. Smith. Martin-Löf’s type theory. S. Abramsky, D. Gabbay, and T. Maibaum (eds.) *Handbook of Logic in Computer Science*, Clarendon Press, 2000, pp. 1-37.
57. G. Plotkin. Building-in equational theories. *Machine Intelligence*, 7, 1972, pp. 73-90.
58. D. Prawitz. *Natural Deduction, a Proof-theoretical Study*. 1965.
59. R. Saillard. Towards explicit rewrite rules in the $\lambda\Pi$ -calculus modulo. *International Workshop on the Implementation of Logics*, 2013.
60. A. Stump, D. Oe, A. Reynolds, L. Hadarean, and C. Tinelli. SMT Proof Checking Using a Logical Framework. *Formal Methods in System Design* 42 (1), 91-118
61. J.R. Slagle. Automatic theorem proving with renamable and semantic resolution. *J. ACM*, 14, 1967, pp. 687-697.
62. B. Wack. *Typage et déduction dans le calcul de réécriture*. Doctoral Thesis, Université Henri Poincaré Nancy 1, 2005.
63. L. Wos, G.A. Robinson, D.F. Carson. Efficiency and completeness of the set of support strategy in theorem proving. *J. ACM*, 12, 1965, pp. 536-541.